FEDERAL UNIVERSITY OYE-EKITI, EKITI STATE

DEPARTMENT OF COMPUTER SCIENCE

CSC 301

(OBJECT ORIENTED PROGRAMMING IN JAVA)

Submitted to: MR. ODUFUWA T.T

BY

S/N	NAME	MATRIC NUMBER
1	BENSON NICHOLAS OLUWAFERANMI	CSC/2023/1067
2	AKEREDOLU OLAREWAJU MICHAEL	CSC/2023/1185
3	ADANLAWO BOLUWAJI EMMANUEL	CSC/2017/1005
4	OLUWATUYI SEGUN JOSIAH	CSC/2023/1147
5	OYEDIKUN-ALLI ALIYAH OLUWATUNMININU	CSC/2023/1165

Table of Content

Introduction

Aim & Objectives

Statement of the problem

Methodology

Algorithm

Implementation

User Guide

Conclusion

Reference

INTRODUCTION

In today's globalized economy, currency conversion has become an essential tool for international trade, travel, and financial transactions. However, obtaining accurate and up-to-date exchange rates often requires users to rely on online websites, banks, or financial institutions, which can be time-consuming and sometimes unreliable.

The Live Currency Converter was created to address this challenge by providing a standalone application that fetches real-time exchange rates from live APIs and enables users to convert currencies instantly with ease. This application eliminates the need to switch between multiple tabs or websites and provides a fast, reliable solution for currency conversion.

With an intuitive graphical interface, support for multiple currencies, flag icons for visual identification, and live exchange rate updates, the Currency Converter simplifies what was traditionally a cumbersome process. This project demonstrates the practical application of Java desktop development using modern libraries and API integration, while emphasizing user experience and accessibility. It reflects the shift toward standalone applications that combine offline-capable user interfaces with live data integration to empower users with accurate financial information at their fingertips.

AIM AND OBJECTIVES

Aim of the Study

The aim of this project is to develop a desktop application that enables users to convert currencies in real-time using live exchange rates from external APIs without relying on web browsers or internet dependencies for the interface.

Objectives of the Study

i. To understand API integration concepts and HTTP requests for fetching live exchange rate data.
ii. To design and develop a user-friendly application with a modern graphical interface for currency conversion.
iii. To implement features such as multi-currency selection, real-time exchange rate fetching, flag icons for visual identification, and instant conversion calculations.
iv. To integrate with the Exchange Rate-API (open.er-api.com) to fetch accurate, up-to-date exchange rates for multiple currencies.
v. To test and validate the application with various currency pairs and conversion amounts to ensure reliable and accurate results.

STATEMENT OF PROBLEM

The process of converting currencies manually or through unreliable online sources has proven to be inefficient and prone to inaccuracies. This often leads to confusion among users conducting international transactions and can result in financial losses due to incorrect exchange rates. The problems identified include:

- **Unreliable Exchange Rate Sources**: Users must rely on multiple websites that may provide outdated or inaccurate exchange rates
- **Time-Consuming Process**: Accessing web browsers and navigating to currency websites wastes valuable time
- **Inconsistent Results**: Different sources provide different rates, leading to confusion about which rate to trust
- **Internet Dependency**: Most currency converters require constant internet connectivity and continuous online access

METHODOLOGY

This chapter presents the methodology adopted in the development of the Live Currency Converter. The project follows a desktop application development approach using Java Swing for the graphical user interface and HTTP client for API integration to fetch live exchange rates.

Unlike projects involving Artificial Intelligence or Complex Algorithms, this application is primarily API-driven and user input-based. However, core software development principles were applied to ensure the system is accurate, responsive, and user-friendly.

Development Approach

- Requirements gathering (reviewing currency conversion needs and API availability)
- Designing the input and output flow (currency selection, amount input, conversion display)
- Implementing the logic using Java and HTTP client libraries
- Integrating with ExchangeRate-API for live exchange rate data
- Testing the application with various currency pairs and conversion amounts
- Packaging the application as a standalone executable JAR file for distribution

Technologies Used

- **Java**: Programming language for backend logic and frontend interface
- **Swing Framework**: For building the graphical user interface (GUI)
- **JSON Processing Library (org.json)**: For parsing and handling JSON responses from the API
- **Maven**: For project dependency management and build automation

Key Features Implemented

1. **Real-time Exchange Rate Fetching**: Automatically fetches live exchange rates from ExchangeRate-API on application startup
2. **Multi-Currency Support**: Displays all available currencies from the API with proper formatting
3. **Flag Icon Display**: Shows country flags alongside currency names for visual identification
4. **Live Conversion Calculation**: Instantly converts between any two currencies using fetched exchange rates
5. **User-Friendly Interface**: Grid-based layout for easy input and clear result display
6. **Error Handling**: Validates user input and handles API failures gracefully
7. **Background Threading**: Fetches rates asynchronously to keep UI responsive
8. **Currency Customization**: Users can select any available currency as source or target

SYSTEM REQUIREMENTS

To ensure optimal performance and compatibility with the Live Currency Converter application, your system should meet the following specifications:

- **Processor**: Any modern AMD, Intel, or other compatible processor
- **RAM:** Minimum of 512 MB installed
- **System Type**: 64-bit operating system is recommended for optimal performance, though 32-bit systems can also run the application
- **Operating System**: Windows 7, 8, 8.1, 10, or 11 (32-bit or 64-bit), macOS, or Linux
- **Java Runtime:** Java 21 or higher installed
- Internet Connection: Required for fetching live exchange rates (one-time per session)

FUTURE IMPROVEMENT

Cryptocurrency Support: Add support for converting between cryptocurrencies (Bitcoin, Ethereum, etc.) alongside traditional fiat currencies by integrating a cryptocurrency API.

Conversion History: Maintain a log of recent conversions that users can quickly access and repeat without re-entering data.

Favorite Currencies: Allow users to save their most frequently used currency pairs for faster access and improved workflow.

Offline Mode: Cache exchange rates locally so conversions can continue even when internet connection is temporarily unavailable.

Copy Result Button: Add a dedicated button to easily copy the conversion result to clipboard for pasting into other applications.

Exchange Rate Trends: Display historical exchange rate changes and trends to help users understand market movements.

ALGORITHM

The following are the processes involved in designing the Currency Converter Application:

Step 1: Start the application

Step 2: Initialize the graphical user interface with currency dropdown boxes, amount input field, and convert button

Step 3: Launch background thread to fetch exchange rates from ExchangeRate-API (open.er-api.com)

Step 4: Parse JSON response containing currency codes and their exchange rates relative to USD base currency

Step 5: Populate "From" and "To" currency dropdown boxes with fetched currencies and flag icons

Step 6: Set default currencies (USD for "From" and NGN for "To")

Step 7: User enters amount to convert in the amount input field

Step 8: User selects source currency from "From" dropdown

Step 9: User selects target currency from "To" dropdown

Step 10: User clicks "Convert" button

Step 11: System validates numeric input

- If input is invalid → Display error message → Return to Step 7
- If input is valid → Proceed to Step 12

Step 12: Retrieve exchange rates for both selected currencies from rates map

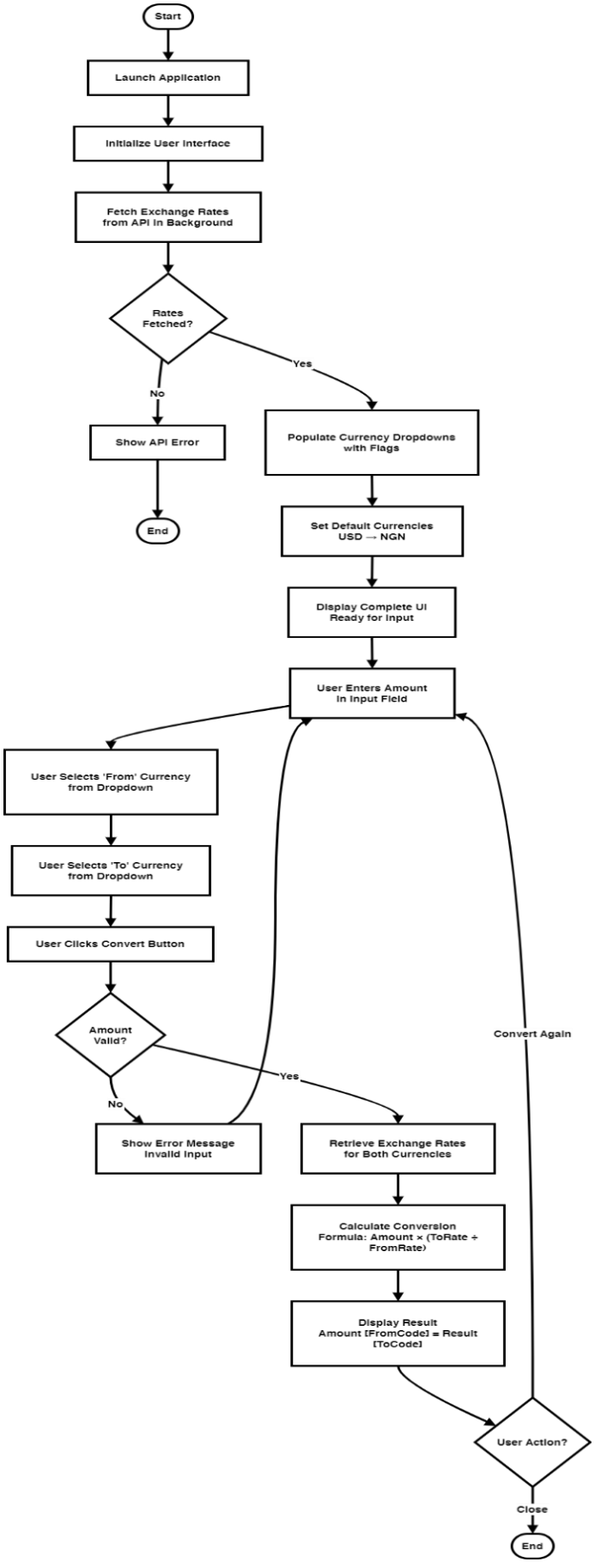
Step 13: Calculate conversion using formula: (Amount) × (Target Rate ÷ Source Rate)

Step 14: Display conversion result with both currency codes and amounts

Step 15: User can perform another conversion by repeating Steps 7-14

Step 16: End or close application

FLOWCHART


Start
Launch Application
Initialize User Interface
Fetch Exchange Rates from API in Background
Rates Fetched?
Yes
No
Show API Error
End
Populate Currency Dropdowns with Flags
Set Default Currencies USD → NGN
Display Complete UI Ready for Input
User Enters Amount in Input Field
User Selects 'From' Currency from Dropdown
User Selects 'To' Currency from Dropdown
User Clicks Convert Button
Amount Valid?
Yes
No
Show Error Message Invalid Input
Retrieve Exchange Rates for Both Currencies
Calculate Conversion Formula: Amount × (ToRate ÷ FromRate)
Display Result Amount [FromCode] = Result [ToCode]
User Action?
Convert Again
Close
End

IMPLEMENTATION

In developing the Live Currency Converter Application, we started by designing and building the interface using Java Swing with Visual Studio Code as our code editor. Before starting the design, we installed Java 21 and configured Maven for dependency management. We then used the ExchangeRate-API integration with Java's built-in HTTP client to enable the core functionalities including real-time exchange rate fetching and currency conversion algorithms.

Application Interface Sections

Title Section: At the top of the application, we placed a clear title "Live Currency Converter (USD Base)" to identify the application purpose and indicate the base currency used for conversions.

From Currency Dropdown: We designed the first dropdown box labeled "From (currency)" where users select their source currency. This dropdown displays currency names with flag icons for visual identification, making it easy for users to identify currencies at a glance. The default selection is USD (United States Dollar).

To Currency Dropdown: Below the "From" dropdown, we implemented the "To (currency)" dropdown where users select their target currency. This section also displays currency names with flag icons and defaults to NGN (Nigerian Naira) for convenience, though users can select any currency available in the API response.

Swap Button: We created a swap button with the icon “⇅” for easy swapping between two selected currencies to see their exchange without reselecting.

Amount Input Field: We created a text input field labeled "Amount" with a default value of 1. This field accepts numeric input that users can modify to specify the quantity of currency they wish to convert. The field is positioned centrally for easy access.

Convert Button: We placed an action button labeled "Convert" that triggers the conversion calculation when clicked. Upon clicking, the application retrieves the exchange rates for both selected currencies and performs the conversion instantly.

Result Display Label: Below the Convert button, we added a result label that displays the conversion output in a clear format: "Result: [Amount] [Source Code] = [Converted Amount] [Target Code]". For example, "Result: 100.00 USD = 150,000.00 NGN". The label initially displays "Result: —" as a placeholder until a conversion is performed.

API Information Section: At the bottom of the application, we included a text area displaying information about the data source: "Live exchange rates provided by open.er-api.com" and "Default base currency: USD". This informs users where the exchange rates originate and ensures transparency about the conversion methodology.

Background Thread for Rate Fetching: We implemented a background thread that automatically fetches exchange rates from the ExchangeRate-API when the application starts. This ensures the application remains responsive while waiting for the API response, and users can see the interface populate with currencies without any freezing or lag.

Currency Renderer with Flag Icons: We created a custom renderer (CurrencyRenderer) that displays both currency codes and flag icons in the dropdown boxes. The flags are scaled to 20x14 pixels and positioned to the left of the currency name, providing visual enhancement and making currency identification faster and more intuitive.

Error Handling: We implemented error handling for invalid numeric inputs and API failures. If users enter non-numeric values, the application displays an error dialog. If the API fails to fetch exchange rates, an error message informs users and provides details about the failure.

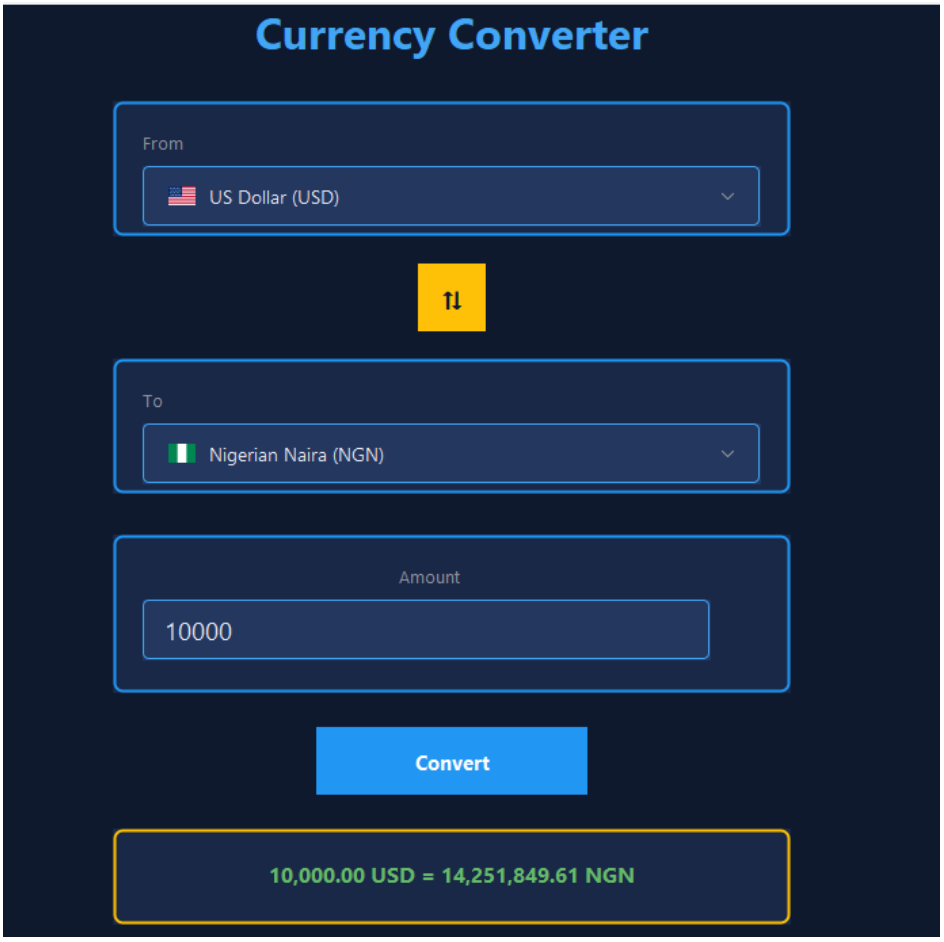
Core Classes

CurrencyConverter.java: The main application class that manages the GUI, API communication, and conversion logic. It contains methods for fetching rates, populating dropdowns, validating inputs, and performing calculations.

CurrencyItem.java: A model class that stores currency information including code, country name, and flag icon. This class is used to populate the dropdown selections.

CurrencyRenderer.java: A custom list cell renderer that extends DefaultListCellRenderer to display currency items with flag icons alongside currency names in the dropdown boxes.

PREVIEW:



CODE SNIPPETS

```java
public class CurrencyConverter extends JFrame {

    private static final long serialVersionUID = 1L;

    // UI Constants - Modern Coherent Palette
    private static final Color PRIMARY_COLOR = new Color(33, 150, 243); // Blue
    private static final Color SECONDARY_COLOR = new Color(66, 165, 245); // Light Blue
    private static final Color ACCENT_COLOR = new Color(255, 193, 7); // Amber
    private static final Color DARK_BG = new Color(15, 25, 45); // Deep Blue-Black
    private static final Color CARD_BG = new Color(25, 40, 70); // Deep Blue
    private static final Color INPUT_BG = new Color(35, 55, 95); // Medium Blue
    private static final Color SUCCESS_COLOR = new Color(102, 187, 106); // Green
    private static final Color WARNING_COLOR = new Color(255, 152, 0); // Orange
    private static final Color TEXT_LIGHT = new Color(230, 230, 240); // Light Text
    private static final Color TEXT_MUTED = new Color(144, 144, 160); // Muted Text

    // Size Constants
    private static final int MIN_CONTENT_WIDTH = 400;
    private static final int MAX_CONTENT_WIDTH = 500;
    private static final int CONTENT_HEIGHT = 100;

    private JComboBox<CurrencyItem> fromBox;
    private JComboBox<CurrencyItem> toBox;
    private JTextField amountField;
    private JLabel resultLabel;
    private JButton convertButton;
    private JLabel loadingLabel;
    private final HttpClient httpClient = HttpClient.newHttpClient();

    // Currency → Rate (relative to USD)
    private final Map<String, Double> ratesMap = new TreeMap<>();

```

CODE SNIPPETS

```java
package com.example;

import javax.swing.*;

public class CurrencyItem {
    public String code;
    public String country;
    public ImageIcon flag;

    public CurrencyItem(String code, String country, ImageIcon flag) {
        this.code = code;
        this.country = country;
        this.flag = flag;
    }

    @Override
    public String toString() {
        return country + " (" + code + ")";
    }
}
```

```java
public class CurrencyRenderer extends DefaultListCellRenderer {

    @Override
    public Component getListCellRendererComponent(JList<?> list, Object value, int index, boolean isSelected,
            boolean cellHasFocus) {

        JLabel label = (JLabel) super.getListCellRendererComponent(list, value, index, isSelected, cellHasFocus);

        if (value instanceof CurrencyItem) {
            CurrencyItem item = (CurrencyItem) value;

            // Format text with currency code highlighted
            String displayText = item.country;
            if (!displayText.equals(item.code)) {
                displayText += " (" + item.code + ")";
            } else {
                displayText = item.code;
            }

            label.setText(displayText);
            label.setIcon(item.flag);
            label.setHorizontalTextPosition(SwingConstants.RIGHT);
            label.setIconTextGap(10);
            label.setFont(new Font("Segoe UI", Font.PLAIN, 14));

            // Better styling for selected items
            if (isSelected) {
                label.setBackground(new Color(76, 175, 80, 100));
                label.setForeground(Color.WHITE);
            }
        }
        return label;
    }
}
```

USER GUIDE

How to use the Live Currency Converter Application?

To run this project, users must have Java 21 or higher installed on their PC with an operating system that supports Java (Windows, macOS, or Linux). An active internet connection is required for the application to fetch live exchange rates.

After launching the Currency Converter application, the following steps should be followed:

Step 1: Launch the Currency Converter application by double-clicking the executable JAR file or running it from the command line

Step 2: The application will display a window with two currency dropdown boxes, an amount input field, and a convert button

Step 3: The application will automatically fetch live exchange rates from the ExchangeRate-API in the background. Wait a moment for the currency dropdowns to populate with all available currencies and flag icons

Step 4: The "From" currency is automatically set to USD (United States Dollar) and the "To" currency is automatically set to NGN (Nigerian Naira). These defaults can be changed at any time by clicking the respective dropdown boxes

Step 5: In the "Amount" field, enter the amount of currency to be converted. The default value is 1, but any numeric value can be entered

Step 6: Click the "From" dropdown box to select a different source currency if conversion from USD is not desired

Step 7: Click the "To" dropdown box to select a different target currency if conversion to NGN is not desired

Step 8: Review the selected currencies - each dropdown displays the currency name alongside a flag icon for easy visual identification

Step 9: Click the "Convert" button to perform the currency conversion

Step 10: The conversion result will be displayed below the convert button in the format: "Result: [Amount] [Source Code] = [Converted Amount] [Target Code]"

Step 11: Another conversion can be performed by changing the amount, selecting different currencies, and clicking "Convert" again

Step 12: Exchange rates are updated each time the application is launched, ensuring the most current conversion rates are always available

Step 13: If an invalid amount is entered (non-numeric), an error dialog will appear. Users should correct the input and try again

Step 14: The application status bar at the bottom display information about the API source for transparency

Step 15: Close the application when finished converting currencies

Tip: You can use the swap button represented by "⇅" to easily swap between the selected currencies to see their rate immediately.

Note: A stable internet connection is required when launching the application to fetch exchange rates. All conversions use USD as the base currency reference provided by the ExchangeRate-API. The application requires a one-time internet connection to fetch rates at startup, but conversion calculations are performed locally and do not require additional internet access.

CONCLUSION

The Live Currency Converter application provides a simple and efficient solution for users to convert currencies in real-time using live exchange rates from a reliable API. By automating the conversion process and fetching current exchange rates automatically, the application eliminates the need for manual lookups or switching between multiple websites.

Its user-friendly interface with flag icons, multi-currency support, and clear result display ensure a seamless user experience. The application uses USD as a base currency reference, allowing users to convert between any two currencies by calculating the relative exchange rates. Real-time API integration ensures users always have access to the most current and accurate exchange rates available.

This tool serves as a valuable resource for travelers, businesses, freelancers, and individuals engaged in international transactions who need quick and reliable currency conversions. The application demonstrates practical desktop application development using modern Java technologies, HTTP client integration, and API consumption. It represents the shift toward standalone applications that combine intuitive user interfaces with real-time data integration to provide users with accurate financial information for informed decision-making.

REFERENCE

- ExchangeRate-API Documentation - https://www.exchangerate-api.com/
- Open ExchangeRate-API - https://open.er-api.com/
- Oracle Java HttpClient Documentation - https://docs.oracle.com/javase/21/docs/api/java.net.http/java/net/http/HttpClient.html
- Java Swing Documentation - https://docs.oracle.com/javase/tutorial/uiswing/
- JSON Processing in Java - https://github.com/stleary/JSON-java
- Wikipedia - Exchange Rate - https://en.wikipedia.org/wiki/Exchange_rate
- Maven Project Management - https://maven.apache.org/